

POSTGRESQL

Accidental TRUNCATE TABLE Recovery

DBA Technical SOP

Purpose	Recover data after an accidental PostgreSQL TRUNCATE TABLE
Primary Decision	Was TRUNCATE committed?
Preferred Committed-Recovery Method	PITR to an isolated instance, then extract and reconcile the table
Validation Basis	PostgreSQL official documentation

Date: 09 September 2026

Contents

1. Executive Summary
2. Key PostgreSQL Behavior
3. Immediate Incident Response
4. Case 1 - TRUNCATE Not Committed
5. Case 2 - TRUNCATE Already Committed: PITR Available
6. Case 3 - No PITR, Recent Backup Available
7. Case 4 - No Backup and No Usable PITR
8. Production Recovery Decision Flow
9. Detailed PITR Recovery Runbook
10. Extracting and Reconciling the Recovered Table
11. Validation Checklist
12. Special Cases and Risks
13. Prevention and Operational Controls
14. DBA Incident Checklist
15. Official PostgreSQL References

1. Executive Summary

An accidental TRUNCATE TABLE is a high-impact database incident because it removes all rows from the target table very quickly. Recovery is straightforward only when the TRUNCATE is still part of an uncommitted transaction. Once committed, PostgreSQL does not provide a built-in command that reverses that committed TRUNCATE.

- If the transaction is still open and uncommitted: issue ROLLBACK immediately.
- If the TRUNCATE committed and PITR is available: recover an isolated PostgreSQL instance to a point immediately before the incident.
- Extract the required table from the recovered instance and carefully reconcile it into current production.
- If PITR is unavailable but a logical/physical backup exists: restore that backup separately and recover what the backup contains.
- If there is no usable backup or recovery chain: supported recovery options are severely limited and some or all data may be unrecoverable.

2. Key PostgreSQL Behavior

2.1 TRUNCATE is transactional

PostgreSQL documents TRUNCATE as transaction-safe with respect to table data. If the surrounding transaction does not commit, the truncation is rolled back.

```
BEGIN;  
  
TRUNCATE TABLE public.customer;  
  
-- Incident noticed before COMMIT  
ROLLBACK;
```

2.2 A committed TRUNCATE cannot be undone with ROLLBACK

After COMMIT, or when autocommit causes the statement to commit, a later ROLLBACK does not reverse the already committed change. At that point, recovery must come from a valid recovery source such as PITR or a backup.

2.3 PITR is cluster-level physical recovery

PostgreSQL continuous archiving combines a physical/base backup with the required WAL history. Recovery replays WAL and can stop at a selected recovery target. PITR restores the cluster state; it is not a direct single-table restore mechanism. A common production pattern is therefore to perform PITR on a separate instance and then logically extract only the required table.

3. Immediate Incident Response

1. Stop further destructive activity. Do not run additional TRUNCATE, DROP, bulk cleanup, or repair commands on the affected objects.
2. Determine whether the developer session/transaction is still open. If it is open and uncommitted, prioritize ROLLBACK.

3. Record the affected database, schema, table name, approximate incident time, application impact, and whether TRUNCATE used CASCADE or RESTART IDENTITY.
4. Preserve current production. Do not overwrite the production data directory with a backup as the first reaction.
5. Identify the recovery sources: latest base backup, WAL archive coverage, backup catalog, logical dumps, replicas, and any external data copies.
6. Establish the exact server time zone before selecting a recovery_target_time.
7. If using PITR, restore to an isolated host/cluster with no application traffic until validation is complete.

4. Case 1 - TRUNCATE Not Committed

This is the fastest and cleanest recovery path.

```
BEGIN;  
TRUNCATE TABLE public.customer;  
  
-- Accident discovered here  
ROLLBACK;  
  
SELECT count(*) AS customer_rows  
FROM public.customer;
```

After ROLLBACK, validate row counts and application behavior. If RESTART IDENTITY was used, sequence restart operations are transactional in supported modern PostgreSQL versions, but sequence behavior should still be validated as part of the incident check.

5. Case 2 - TRUNCATE Already Committed: PITR Available

For a committed TRUNCATE, PITR is normally the strongest supported recovery method when a suitable base backup and the required WAL sequence are available.

5.1 Example incident timeline

```
14:00:00 Database healthy  
14:05:10 Developer executes TRUNCATE  
14:05:11 TRUNCATE commits  
14:10:00 DBA receives incident
```

Recover the isolated instance to a target immediately before the destructive transaction. The exact target must be chosen from verified server/database timing, not guessed.

```
# Example recovery settings - adapt to your backup tooling and PostgreSQL version  
  
restore_command = 'cp /mnt/wal_archive/%f %p'  
recovery_target_time = '2026-09-09 14:05:09-05'  
recovery_target_action = 'promote'
```

Create the required recovery signal/configuration according to the PostgreSQL version and backup tool in use. Do not copy this example blindly into production; paths, permissions, timelines, and recovery targets must be validated.

6. Case 3 - No PITR, Recent Backup Available

Restore the most recent usable backup to a separate PostgreSQL environment. Then extract the affected table or rows and reconcile them into production.

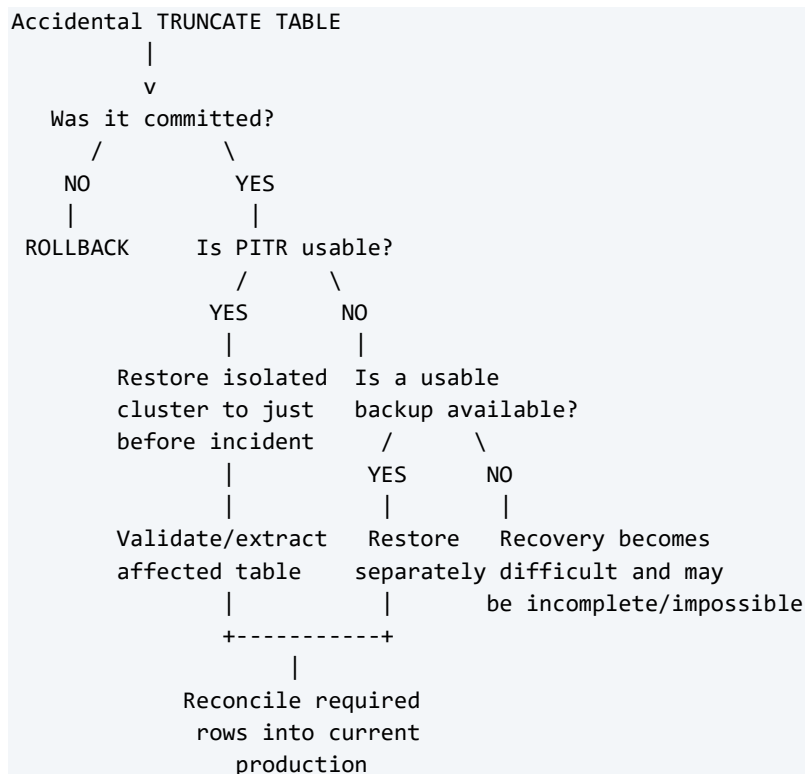
- A logical `pg_dump` backup can restore the data represented by that dump, but it is not itself a PITR base backup.
- A physical backup without the required WAL cannot normally be advanced to the exact pre-incident moment.
- Any changes after the backup and before `TRUNCATE` may be missing from the recovered copy.
- Validate referential integrity and dependent tables before reloading data.

7. Case 4 - No Backup and No Usable PITR

PostgreSQL has no supported built-in 'undo committed `TRUNCATE`' command. The live `pg_wal` directory is primarily part of PostgreSQL crash recovery and WAL processing; it should not be treated as a recycle bin or a guaranteed source for reconstructing a committed truncated table.

- Check whether another trusted copy exists: reporting database, replica snapshot, export, downstream warehouse, application cache, or business source system.
- Preserve evidence and avoid aggressive filesystem-level experiments on the only production copy.
- Escalate to the incident owner and communicate the recoverable data window clearly.
- If third-party forensic recovery is considered, perform it only on copies and treat the outcome as uncertain, not as a standard PostgreSQL recovery method.

8. Production Recovery Decision Flow



9. Detailed PITR Recovery Runbook

Step 1 - Confirm incident scope

Identify database, schema, table, incident time, transaction state, CASCADE usage, identity/sequence impact, and dependent objects.

Step 2 - Confirm backup chain

Verify a base backup exists before the target time and confirm continuous WAL coverage from that backup through the desired recovery point.

Step 3 - Build an isolated recovery target

Use a separate server, VM, container, or isolated PostgreSQL cluster. Match the PostgreSQL major version and required extensions/libraries.

Step 4 - Restore the base backup

Restore using the approved backup tool/process. Ensure PostgreSQL ownership and permissions are correct.

Step 5 - Configure WAL retrieval

Set `restore_command` or the equivalent supported mechanism used by your backup solution so archived WAL can be retrieved.

Step 6 - Set the recovery target

Set `recovery_target_time`, a named restore point, or another supported target. For this incident, the target should be before the TRUNCATE commits.

Step 7 - Start recovery

Start PostgreSQL and monitor logs. Confirm WAL replay progresses and recovery stops at the intended target.

Step 8 - Isolate and validate

Do not allow normal application traffic. Validate database state, target table row counts, key business rows, constraints, and sequences.

Step 9 - Export the recovered table

Use `pg_dump` or controlled SQL/COPY to export the required table/data from the recovered instance.

Step 10 - Reconcile into production

Compare recovered data with current production and load only the required data using a controlled transaction and business-approved merge strategy.

Step 11 - Post-recovery validation

Validate row counts, constraints, application checks, audit records, and business totals. Keep incident evidence and recovery logs.

10. Extracting and Reconciling the Recovered Table

10.1 Validate the recovered copy

```
SELECT count(*) AS recovered_rows
FROM public.customer;

SELECT min(customer_id) AS min_id,
       max(customer_id) AS max_id
FROM public.customer;
```

10.2 Export the table

```
pg_dump \
-d production_db \
-t public.customer \
-Fc \
-f customer_recovered.dump
```

Custom format (-Fc) is suitable for pg_restore and allows selective restore behavior. For production reconciliation, restoring directly over the live table may be unsafe if new rows or related changes have occurred after the incident.

10.3 Safer merge pattern

- Restore the recovered data into a staging database or staging table first.
- Compare primary keys and business keys against current production.
- Identify rows that are missing, rows recreated after the incident, and rows whose values changed.
- Obtain application/business approval for conflict handling.
- Insert/merge only the approved rows inside a controlled transaction.
- Validate sequences/identity values, foreign keys, triggers, and downstream systems.

11. Validation Checklist

Validation	Evidence	Status
Correct recovery target reached	PostgreSQL recovery logs and target timestamp	☐
Target table contains expected rows	COUNT, key ranges, business sample queries	☐
No unintended dependent tables were missed	Dependency/FK review, CASCADE check	☐
Sequences are valid	Sequence last value versus maximum key	☐
Recovered data reconciled safely	Staging comparison and approved merge	☐
Application works normally	Functional validation	☐
Backup/PITR chain remains healthy	Post-incident backup and restore test	☐

12. Special Cases and Risks

12.1 TRUNCATE ... CASCADE

TRUNCATE CASCADE can include tables that have foreign-key relationships with the target. Always identify every table affected before recovery. Recovering only the originally named table can leave business data inconsistent.

12.2 RESTART IDENTITY

TRUNCATE ... RESTART IDENTITY restarts owned sequences. When restoring data, validate sequence values before reopening writes to avoid duplicate-key or unexpected key-generation problems.

12.3 Recovery target time and time zones

A recovery target selected in the wrong time zone can replay the destructive transaction or stop too early. Capture PostgreSQL/server time zone, log timestamps, and the exact incident timestamp.

12.4 Do not roll back the whole production system unnecessarily

If production continued receiving valid transactions after the TRUNCATE, replacing production with a PITR copy from before the incident would also discard those valid later transactions. Recovering separately and extracting only the affected data usually reduces collateral data loss.

13. Prevention and Operational Controls

- Restrict TRUNCATE privileges using least privilege. Developers should not have destructive production privileges unless required.
- Use controlled change tickets and peer review for production data-removal operations.
- Require explicit transactions for approved high-risk maintenance where practical, with validation before COMMIT.
- Maintain tested base backups and continuous WAL archiving for systems that require PITR.
- Run scheduled restore drills. A backup is not proven until recovery has been tested.
- Use database/audit logging appropriate to the environment so the exact incident time and actor can be established.
- For large or critical tables, define an incident-specific recovery runbook and RPO/RTO before an outage occurs.

14. DBA Incident Checklist

- ☐ Identify affected database/schema/table.
- ☐ Confirm whether transaction is still open.
- ☐ ROLLBACK immediately if TRUNCATE is uncommitted.
- ☐ Record exact incident timestamp and server time zone.
- ☐ Check whether CASCADE or RESTART IDENTITY was used.
- ☐ Confirm base backup and WAL archive coverage.
- ☐ Preserve current production state.
- ☐ Restore to an isolated recovery environment.
- ☐ Recover to immediately before the destructive commit.
- ☐ Validate recovered rows and dependencies.
- ☐ Export to staging, compare, and reconcile.
- ☐ Validate sequences, constraints, application, and business totals.
- ☐ Document RCA and implement preventive controls.

15. Official PostgreSQL References

- [PostgreSQL 18 - Server Administration / Backup and Restore](#)
- [PostgreSQL - TRUNCATE command](#)
- [PostgreSQL - Continuous Archiving and Point-in-Time Recovery \(PITR\)](#)

- [PostgreSQL - pg_dump](#)

Operational note: Commands in this SOP are examples. Before production execution, adapt paths, ports, backup-tool syntax, PostgreSQL version details, recovery target, permissions, and application reconciliation logic to the actual environment.